PRELIMINARY EXAMINATIONS 2026

Course 2: Programming Fundamentals with C++

A Comprehensive Academic Reference on Program Compilation, Memory Architectures, Dynamic Logic, and Object-Oriented Software Design in C++

COURSE CODE
CS-102

ACADEMIC YEAR
2026

PUBLISHER
Preliminary Examinations Press

TARGET AUDIENCE
Undergraduate Science & Engineering Majors

Table of Contents

MODULE 1

Introduction to Programming in C++

1.1 Structure of a C++ Program

C++ is a high-performance, statically-typed, compiled, general-purpose programming language developed by Bjarne Stroustrup in 1979 at Bell Labs as an extension of the C language. By adding object-oriented support, C++ provides programmers with low-level hardware control alongside high-level abstractions. A standard C++ program contains three fundamental structural blocks: preprocessor directives (for including headers), namespace declarations (to organize code symbols), and the `main()` function where the OS transfers execution control.

Let us examine the anatomy of a standard, compilable C++ program:

```
#include <iostream> // Preprocessor directive to include input-output stream decla

// Using standard namespace to avoid typing std:: repeatedly
using namespace std;

// The main function where program execution begins
int main() {
    // cout represents the console output stream.
    // The << operator inserts the string into the stream.
    cout << "Hello, Preliminary Examinations 2026!" << endl;

    return 0; // Return value indicating successful termination to the OS
}
```

1.2 The C++ Compilation Pipeline

C++ source code is not executed directly by a virtual machine. Instead, it is translated into hardware-specific binary instructions. This process is called the **Compilation Pipeline** and consists of four main phases:

1. **Preprocessing:** The preprocessor processes lines starting with `#`. It copies included headers (e.g., contents of `<iostream>`) directly into the source file and expands macros (e.g., `#define`). The output is an expanded translation unit.

2. **Compilation:** The compiler translates the expanded C++ source code into assembly code specific to the target CPU architecture. It checks syntax, type alignments, and logical structures.
3. **Assembly:** The assembler translates the assembly instructions into binary object code (machine instructions). The output files typically have `.obj` (Windows) or `.o` (Unix) extensions.
4. **Linking:** The linker combines the generated object files with external library object files (like runtime libraries) to create a single, standalone executable binary file (`.exe` or `.out`).

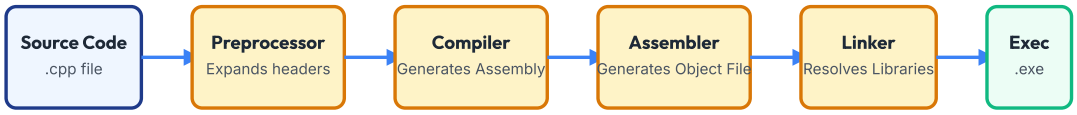

Figure 1.1: The sequential processing pipeline of a C++ source file compilation.

1.3 Variables, Data Types, and Memory Footprints

Variables are named storage locations in physical RAM. C++ is a strongly-typed language, meaning every variable must be declared with a data type. This type determines the variable's size in memory and its value range.

Data Type	Keyword	Size (Typical)	Range Value Limits
Integer	`int`	4 Bytes	-2,147,483,648 to 2,147,483,647
Character	`char`	1 Byte	-128 to 127 (or ASCII character representations)
Boolean	`bool`	1 Byte	`true` (1) or `false` (0)
Floating Point	`float`	4 Bytes	~3.4e±38 (7 decimal digits precision)
Double Floating Point	`double`	8 Bytes	~1.7e±308 (15 decimal digits precision)

Constants: Variables whose values cannot be changed after initialization. They can be defined using the `const` keyword or the preprocessor directive `#define`:

```
const double PI = 3.141592653589793; // Declares a read-only variable in memory
#define MAX_BUFFER_SIZE 1024         // Textual substitution before compilation
```

1.4 Input/Output and Operator Precedence Rules

C++ uses streams for input and output operations:

- `std::cout`: Standard character output stream (console). Used with the stream insertion operator (`<<`).
- `std::cin`: Standard character input stream (keyboard). Used with the stream extraction operator (`>>`).

```
int age;
cout << "Enter your age: ";
cin >> age; // Reads user input and stores it in the age variable
cout << "You are " << age << " years old." << endl;
```

Operators and Expression Precedence

C++ expressions are evaluated based on operator precedence and associativity rules. Below is a summary of the precedence hierarchy:

1. **Postfix Operators:** Increment/Decrement (`x++`, `x--`) and function calls.
2. **Unary Operators:** Prefix Increment/Decrement (`++x`, `--x`), logical NOT (`!`), and positive/negative signs (`+`, `-`).
3. **Multiplicative Operators:** Multiplication (`*`), division (`/`), and modulo (`%`).
4. **Additive Operators:** Addition (`+`) and subtraction (`-`).
5. **Relational & Logical Operators:** Comparison (`<`, `>`, `==`, `≠`), followed by logical AND (`&&`) and logical OR (`||`).

1.5 Explicit Type Casting and Debugging Basics

Type Casting: Converting a variable from one data type to another.

- *Implicit Casting:* Automatic conversion by the compiler when converting a smaller type to a larger type (e.g., `int` to `double`).
- *Explicit Casting:* Manual conversion by the programmer. The standard C++ syntax is `static_cast<new_type>(expression)`:

```
int total = 10;
int count = 4;
// Casts total to double to prevent integer division, yielding 2.5
double average = static_cast<double>(total) / count;
```

Debugging Basics

Debugging is the process of identifying and resolving bugs in code. Errors generally fall into three categories:

1. **Syntax Errors:** Violations of C++ grammar rules (e.g., missing a semicolon or mismatching parentheses). These prevent the code from compiling.
2. **Runtime Errors:** Errors that occur during program execution, causing the program to crash (e.g., dividing by zero or accessing invalid memory).
3. **Linker Errors:** Occur when the compiler cannot locate the definition of a declared function or variable (e.g., misspelling a function name).
4. **Logical Errors:** The program runs without crashing but produces incorrect outputs due to an error in the programmer's logic.

Module 1 - Exercises & Review Tasks

1 Debugging Task: Syntax and Type Errors

Find and correct all compile errors in the following C++ code:

```
#include <iostream>
#define TAX_RATE = 0.05;

int main() {
    int price = 19.99
    const int discount = 2;
    discount = 3;
    cout << "Final price: " << price * TAX_RATE - discount;
    return "success";
}
```

2 Guess the Output: Post vs Prefix Operators

Analyze the execution flow and state the exact output of this program:

```
#include <iostream>
int main() {
    int a = 5;
    int b = a++;
    int c = ++a;
    std::cout << "a: " << a << ", b: " << b << ", c: " << c << std::en
    return 0;
}
```

3 Theory Review Questions

Answer the following conceptual questions in detail:

- Describe the four main phases of the C++ compilation pipeline. What file extensions are produced in each phase?
- What is the difference between implicit and explicit type casting? Provide C++ code snippets for both.

- Discuss the differences between `#define` macro constants and `const` variables. Why is `const` preferred in modern C++?
- What is the role of the `using namespace std;` statement? What are the potential conflicts of using it globally?

Module 1 - Practical Lab Work

Lab 1.1: Standard Conversions Calculator

Write a complete C++ program that reads a temperature value in Fahrenheit from standard input and outputs the equivalent temperature in Celsius. Use the formula: $C = (F - 32) \times 5 / 9$.

Requirements: Ensure the output contains decimal precision by using explicit type casting or decimal literals.

MODULE 2

Decision Making & Iteration

2.1 Conditional Branching: If-Else structures

Conditional statements allow a program to execute different blocks of code depending on whether a condition evaluates to true or false. In C++, these are implemented using `if`, `else if`, and `else`:

```
int score = 85;
if (score ⩾ 90) {
    std::cout << "Grade: A" << std::endl;
} else if (score ⩾ 80) {
    std::cout << "Grade: B" << std::endl; // Executed if score is 85
} else {
    std::cout << "Grade: F" << std::endl;
}
```

2.2 Switch-Case Statements & Jump Tables

A `switch` statement evaluates a single expression and matches it against multiple case constants. It is often more efficient and readable than an nested if-else chain. Compilers typically optimize switch statements by creating a **Jump Table**, which allows direct branch selection without sequential comparisons.

Critical Concept: Fall-through. When a matching case is found, execution continues into subsequent cases unless interrupted by a `break` statement. While sometimes used intentionally, missing a break statement is a common source of logical bugs.

```
char operation = '+';
switch (operation) {
    case '+':
        std::cout << "Addition Selected" << std::endl;
        break; // Interrupts fall-through
    case '-':
        std::cout << "Subtraction Selected" << std::endl;
        break;
    default:
        std::cout << "Unknown Operation" << std::endl;
}
```

2.3 Iteration Control: Loops

Loops allow a program to repeat a block of code while a condition remains true. C++ provides three loop structures:

- **while Loop:** Evaluates the loop condition before executing the loop body. The body may not run at all if the condition is initially false.
- **do-while Loop:** Evaluates the loop condition after executing the loop body. This guarantees the body runs at least once.
- **for Loop:** Combines initialization, condition testing, and iteration update into a single line. This loop is ideal for repeating a block of code a set number of times.

```
// for loop printing numbers from 0 to 4
for (int i = 0; i < 5; i++) {
    std::cout << i << " ";
}
```

2.4 Nested Loops & Pattern Printing Algorithms

Nested loops are loops placed inside other loops. They are often used to manipulate multi-dimensional grids, print geometric text patterns, or perform nested calculations.

Pattern Algorithm: Half Pyramid

To print a half pyramid of stars, an outer loop controls the rows, while an inner loop prints the stars for each row:

```
for (int i = 1; i ⩽ 5; i++) {
    for (int j = 1; j ⩽ i; j++) {
        std::cout << "*";
    }
    std::cout << "\n";
}
```

2.5 Variable Scope, Shadowing & Loop Interruptions

Variable Scope: The region of a program where a variable is accessible.

- *Global Scope:* Variables declared outside any function, accessible throughout the entire file.
- *Local Scope:* Variables declared inside a function or loop block `{}`, accessible only within that block.

Variable Shadowing: Occurs when a variable declared within an inner scope has the same name as a variable in an outer scope. The inner variable "shadows" or hides the outer one.

Loop Interruptions:

- `break`: Immediately terminates the loop execution and transfers control to the statement following the loop.
- `continue`: Skips the remaining statements in the current iteration of the loop and moves to the next evaluation cycle.

Module 2 - Exercises & Review Tasks

1

Debugging Task: Unintentional Fall-through

Find the logic error in this program designed to execute a basic calculator:

```
#include <iostream>
int main() {
    int x = 5, y = 3;
    char op = '-';
    switch (op) {
        case '+': std::cout << x + y;
        case '-': std::cout << x - y;
        default: std::cout << "Invalid";
    }
    return 0;
}
```

2

Guess the Output: Loop Control Statements

Determine the exact console output of this code snippet:

```
#include <iostream>
int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) continue;
        if (i == 5) break;
        std::cout << i << " ";
    }
    return 0;
}
```

3

Theory Review Questions

Answer the following conceptual questions in detail:

- Differentiate between `if-else` ladders and `switch-case` statements. In what scenarios does the compiler optimize switch statements using a Jump Table?

- What is variable shadowing in C++? Provide a minimal code snippet illustrating a local variable shadowing a global variable.
- Compare the behaviors of the `break` and `continue` statements inside loops.
- Explain why the condition in a `do-while` loop is evaluated at the end of the loop. Give an example scenario where a `do-while` loop is preferred over a standard `while` loop.

Module 2 - Practical Lab Work

Lab 2.1: Floyd's Triangle Pattern

Write a C++ program that reads an integer N from the user and prints Floyd's Triangle with N rows. Floyd's triangle is a right-angled triangle filled with consecutive numbers starting from 1.

Example output for N=4:

```
1
2 3
4 5 6
7 8 9 10
```

MODULE 3

Functions, Arrays & Strings

3.1 Function Declarations and Parameter Passing

A **Function** is a reusable block of code that performs a specific task. Using functions improves code modularity and readability. A function must be declared (prototype) before it is called:

```
// Function prototype
int add(int x, int y);

int main() {
    int result = add(5, 7); // Function call
    return 0;
}

// Function definition
int add(int x, int y) {
    return x + y;
}
```

Parameter Passing: Value vs Reference

C++ supports two primary methods for passing parameters to functions:

1. **Pass by Value:** The function receives a copy of the argument. Modifications to the parameter inside the function do not affect the original variable.
2. **Pass by Reference:** The function receives a reference to the original argument. Modifications to the parameter inside the function change the original variable directly. Passed variables are declared with the address character `&`:

```
// Pass-by-reference swap function
void swap(int &x, int &y) {
    int temp = x;
    x = y;
    y = temp;
}
```

3.2 Function Overloading & Recursive Algorithms

Function Overloading: Allows multiple functions to share the same name, provided they have different parameter lists (different number of arguments or different argument data types).

```
int printValue(int x);
double printValue(double x); // Overloaded name with different data type
```

Recursion: A programming technique where a function calls itself to solve a smaller instance of the same problem. A recursive function must contain a **Base Case** to terminate the recursion; otherwise, it will execute indefinitely, leading to a **Stack Overflow** error.

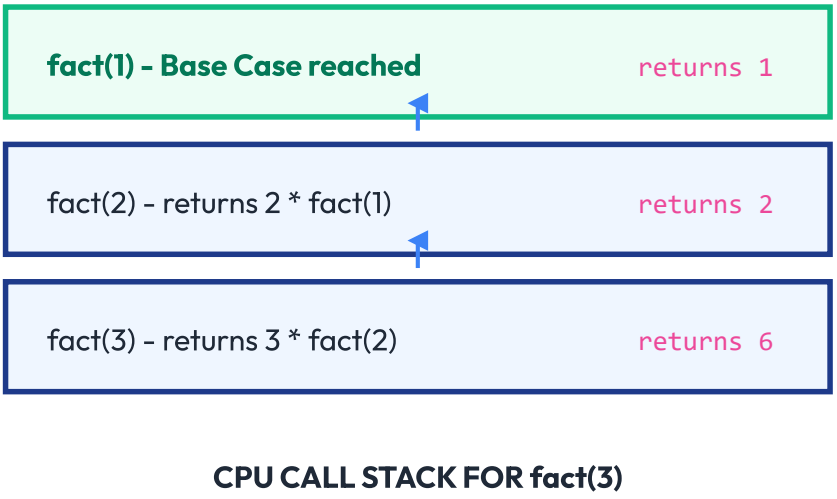

Figure 3.1: Call stack frame trace for a recursive factorial function call.

3.3 Array Memory Structures

An **Array** is a collection of elements of the same data type stored in contiguous memory locations.

- **1D Array:** Declared using square brackets. Element access uses 0-based indexing:

```
int scores[5] = {90, 85, 77, 92, 88};
std::cout << scores[0]; // Prints the first element (90)
```

- **2D Array (Matrix):** Represented as a grid with rows and columns:

```
int matrix[2][3] = {
    {1, 2, 3}, // Row 0
    {4, 5, 6}  // Row 1
};
std::cout << matrix[1][2]; // Prints element at Row 1, Column 2 (6)
```

3.4 Character Arrays vs std::string Classes

C++ provides two ways to work with text strings:

1. **C-Style Strings:** Arrays of characters ending with a null terminator character (`'\0'`). They require careful bounds checking to prevent buffer overflow vulnerabilities.
2. **std::string Class:** A standard library class that manages string memory automatically, resizing dynamically as needed.

```
char name_old[] = "John"; // Length is 5 bytes due to implicit '\0'
std::string name_new = "John"; // Dynamically managed string class
```

3.5 Searching, Sorting, and Big O Complexity Analysis

Searching and sorting are fundamental algorithms in computer science:

- **Linear Search:** Compares the search target against each element in the array sequentially. Time complexity is $O(n)$.
- **Binary Search:** A faster search algorithm that works only on sorted arrays. It repeatedly divides the search interval in half. Time complexity is $O(\log n)$.
- **Bubble Sort:** A simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Time complexity is $O(n^2)$.

Introduction to Big O Notation

Big O notation is used to describe the efficiency of an algorithm in terms of execution time or memory space as the input size (n) grows. Common complexity categories include:

- $O(1)$: Constant time. Execution time remains unchanged regardless of input size.
- $O(\log n)$: Logarithmic time. Execution time grows logarithmically relative to input size (e.g., Binary Search).

- $O(n)$: Linear time. Execution time grows linearly with input size (e.g., Linear Search).
- $O(n^2)$: Quadratic time. Execution time grows quadratically with input size (e.g., Bubble Sort, nested loops).

Module 3 - Exercises & Review Tasks

1

Debugging Task: Infinite Recursion

Identify and correct the bug causing a Stack Overflow in this recursive program:

```
#include <iostream>
int sumTo(int n) {
    if (n == 0) return 0;
    return n + sumTo(n - 1);
}
int main() {
    std::cout << sumTo(-5);
    return 0;
}
```

2

Guess the Output: Array Boundary Tracing

Determine the output printed on the console:

```
#include <iostream>
void modifyArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2;
    }
}
int main() {
    int data[] = {1, 2, 3};
    modifyArray(data, 3);
    std::cout << data[0] << " " << data[1] << " " << data[2];
    return 0;
}
```

3

Theory Review Questions

Answer the following conceptual questions in detail:

- Compare Pass-by-Value and Pass-by-Reference in terms of execution speed, memory footprint, and logical side effects.

- Explain the structural changes in the call stack when a recursive function is executing. What causes a "Stack Overflow"?
- Differentiate between C-style character arrays and the `std::string` class in C++. Which is more secure and why?
- Explain the operational logic of Binary Search. Why must the input array be sorted before search initialization? What is its Big O time complexity?

Module 3 - Practical Lab Work

Lab 3.1: Binary Search Implementation

Write a C++ program that initializes a sorted array of 10 integers. Ask the user to input a search key, and use the Binary Search algorithm to find the index of the key in the array. If the key is not found, output a message indicating so.

Requirement: Print the lower, middle, and upper bounds of the search interval during each iteration to trace execution flow.

MODULE 4

Memory Management & Pointers

4.1 Memory Layout of C++ Executables

A running C++ program uses physical RAM organized into distinct memory segments:

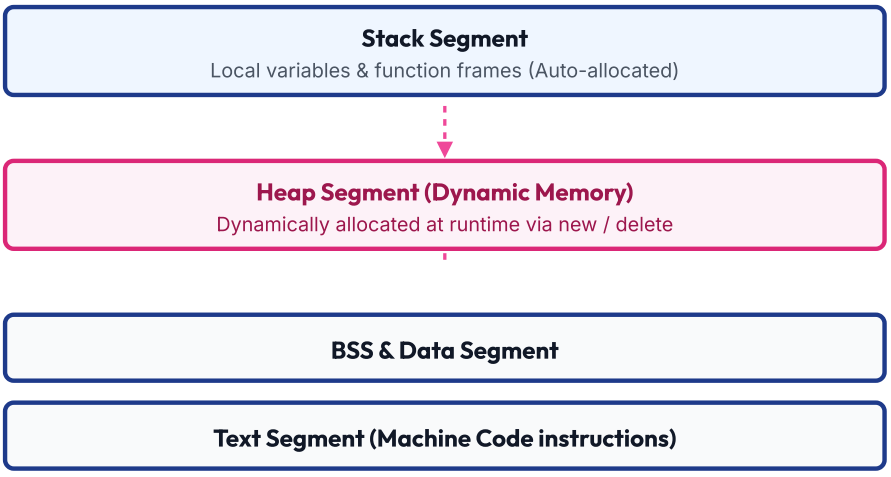

Figure 4.1: Structural layout of a C++ program memory segments in RAM.

4.2 Address Operators & Pointer Basics

Every variable is stored at a specific memory address in RAM. C++ allows direct access to these addresses using pointers:

- **Address-of Operator (`&`):** Returns the memory address of a variable.
- **Pointer:** A variable that stores the memory address of another variable.
- **Dereference Operator (`*`):** Accesses the value stored at the memory address pointed to by a pointer.

```
int count = 42;
int *ptr = &count; // ptr holds the memory address of count
std::cout << ptr;  // Prints memory address (e.g. 0x7ffd58)
std::cout << *ptr; // Dereferences ptr to print count's value (42)
```

4.3 Pointer Arithmetic & Array Address Duality

Pointer arithmetic is performed based on the size of the data type the pointer points to. For example, incrementing an integer pointer (`ptr++`) adds 4 bytes to the stored address (assuming a 4-byte `int`).

Arrays & Pointers: An array name acts as a constant pointer pointing to the first element of the array. The expression `arr[i]` is evaluated by the compiler as `*(arr + i)`.

```
int arr[3] = {10, 20, 30};
int *p = arr; // Equivalent to int *p = &arr[0];
cout << *p;     // Prints 10
cout << *(p+1); // Prints 20
```

4.4 Dynamic Allocation and Pointers (new/delete)

Dynamic memory allocation allows a program to request memory from the heap during runtime. This memory remains allocated until it is explicitly freed by the programmer.

- `new`: Allocates memory on the heap.
- `delete`: Frees memory allocated by `new`.
- `new[]` / `delete[]`: Used to allocate and free arrays on the heap.

```
int *d_val = new int(10); // Allocates a single int on the heap
delete d_val;            // Frees the allocated memory

int *d_array = new int[5]; // Allocates an array of 5 ints on the heap
delete[] d_array;          // Frees the allocated array memory
```

> **Critical Concept: Memory Leaks**
>
> A **Memory Leak** occurs when dynamically allocated heap memory is no longer referenced but has not been freed using `delete`. If a program leaks memory repeatedly, it may consume all available system RAM, causing the operating system to terminate the program.

4.5 Null, Dangling, and Wild Pointers

Working with raw pointers requires careful memory management to avoid bugs:

- **Wild Pointer:** A pointer that has been declared but not initialized. It points to a random memory address. Accessing it can cause undefined behavior.
- **Dangling Pointer:** A pointer that points to a memory address that has already been deallocated (freed).
- **Null Pointer:** A pointer initialized to point to nowhere (using `nullptr` in modern C++). It is safe to check if a pointer is null before dereferencing it.

Module 4 - Exercises & Review Tasks

1

Debugging Task: Dangling Dereference

Explain why this program causes a crash or undefined behavior, and write the corrected code:

```
#include <iostream>
int* getAddress() {
    int temp = 100;
    return &temp;
}
int main() {
    int *ptr = getAddress();
    std::cout << *ptr;
    return 0;
}
```

2

Guess the Output: Pointer Arithmetic

Predict the output printed by this program:

```
#include <iostream>
int main() {
    int arr[] = {10, 20, 30, 40};
    int *p = arr;
    p += 2;
    std::cout << *p << " " << *(p - 1);
    return 0;
}
```

3

Theory Review Questions

Answer the following conceptual questions in detail:

- Differentiate between Stack and Heap memory allocations in C++ in terms of execution speed, lifetime of variables, and allocation capacity.
- What is a memory leak? Provide a code sample showing how a memory leak occurs, and write its corrected version.

- Explain the differences between Pointers and References in C++. Can a reference be declared without initialization?
- Define the terms: Wild Pointer, Dangling Pointer, and Null Pointer. Write a C++ code block demonstrating a dangling pointer scenario.

Module 4 - Practical Lab Work

Lab 4.1: Dynamic Int Array Manager

Write a C++ program that asks the user to input the size of an array. Dynamically allocate an integer array of that size on the heap. Prompt the user to fill the array, calculate and print the average value of its elements, and then free the allocated memory.

Requirement: Set the pointer to `nullptr` after deallocating to prevent dangling pointer bugs.

MODULE 5

Object-Oriented Programming

5.1 Classes, Objects, and Access Specifiers

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data (attributes) and code (methods).

- **Class:** A user-defined blueprint or template used to create objects.
- **Object:** An instance of a class.
- **Access Specifiers:** Define the visibility of class members:
  - `public`: Members are accessible from outside the class.
  - `private`: Members are accessible only from within the class. This is the default access level.
  - `protected`: Members are accessible within the class and its derived classes.

```
class Student {
private:
    std::string name; // Accessible only within the class
public:
    void setName(std::string s_name) {
        name = s_name;
    }
};
```

5.2 Constructors, Member Initializers, and Destructors

Constructors and destructors are special member functions called automatically during an object's lifecycle:

- **Constructor:** Initializes class member variables when an object is created. Constructors share the same name as the class and do not have a return type. They can be overloaded.
- **Destructor:** Cleans up resources (such as closing files or freeing heap memory) when an object goes out of scope or is deleted. Destructors share the same name as the class, prefixed with a tilde (`~`), and cannot take parameters or return values.

```
class Buffer {
private:
    int *data;
public:
    Buffer(int size) {
        data = new int[size]; // Resource allocation in constructor
    }
    ~Buffer() {
        delete[] data;       // Resource cleanup in destructor
    }
};
```

5.3 this Pointer, Static Members & Friend Classes

this Pointer: An implicit pointer available within non-static member functions. It points to the object that invoked the function.

Static Members: Static variables and functions are shared across all instances of a class. They belong to the class itself rather than individual objects.

Friend Functions & Classes: Declaring a function or class as a `friend` inside another class grants it access to the private and protected members of that class.

5.4 Encapsulation, Abstraction & Basic Inheritance

Encapsulation: Bundling data and methods that operate on that data into a single unit (class) and restricting access to the inner workings (data hiding).

Abstraction: Hiding complex implementation details and showing only the essential features of an object.

Inheritance: Allows a new class (derived class) to inherit attributes and methods from an existing class (base class), promoting code reuse.

```
class Animal {
public:
    void eat() { cout << "Eating..." << endl; }
};
class Dog : public Animal { // Dog inherits eat() from Animal
public:
    void bark() { cout << "Barking..." << endl; }
};
```

5.5 Virtual Functions, VTables & Polymorphism

Polymorphism: The ability of different classes to respond to the same function call in different ways.

- *Compile-time Polymorphism:* Implemented using function overloading or operator overloading.
- *Runtime Polymorphism:* Implemented using inheritance and **Virtual Functions**. Declaring a base class function as `virtual` tells the compiler to resolve calls at runtime based on the object's actual type.

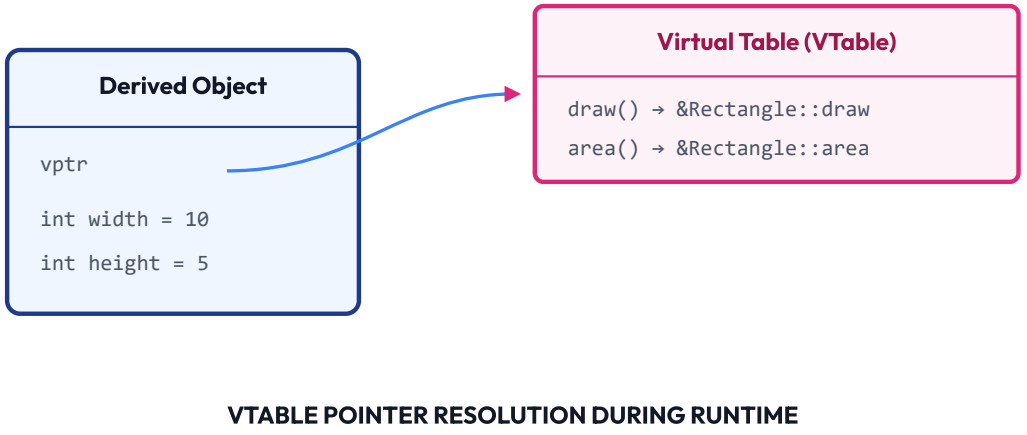

Figure 5.1: Virtual Table (VTable) lookup resolution for runtime polymorphism.

Pure Virtual Functions & Abstract Classes

A **Pure Virtual Function** is a function declared in a base class with no implementation, denoted by assigning it to 0:

```
class Shape {
public:
    virtual double getArea() = 0; // Pure virtual function declaration
};
```

A class containing at least one pure virtual function is an **Abstract Class**. Abstract classes cannot be instantiated directly; they serve as interfaces that derived classes must implement.

Module 5 - Exercises & Review Tasks

1

Debugging Task: Missing Virtual Destructor

Explain the memory leak in this OOP hierarchy and correct it:

```
#include <iostream>
class Base {
public:
    Base() { }
    ~Base() { std::cout << "~Base\n"; }
};
class Derived : public Base {
private:
    int *arr;
public:
    Derived() { arr = new int[100]; }
    ~Derived() { delete[] arr; std::cout << "~Derived\n"; }
};
int main() {
    Base *ptr = new Derived();
    delete ptr; // Triggers destructor
    return 0;
}
```

2

Guess the Output: Virtual Method Resolution

Determine the output printed on execution:

```
#include <iostream>
class Animal {
public:
    virtual void makeNoise() { std::cout << "Generic Noise "; }
};
class Dog : public Animal {
public:
    void makeNoise() override { std::cout << "Woof "; }
};
int main() {
    Animal *a = new Dog();
    a->makeNoise();

    Animal b = Dog();
    b.makeNoise(); // Note: Object slicing occurs here!

    delete a;
    return 0;
}
```

3

Theory Review Questions

Answer the following conceptual questions in detail:

- List the four primary pillars of Object-Oriented Programming (OOP) and write a 1-sentence definition for each.
- What is the function of a Constructor? Explain the order in which base class and derived class constructors are executed in an inheritance chain.
- Explain the concept of Runtime Polymorphism. Why must the destructor of a base class containing virtual functions be declared as `virtual`?
- What is a Pure Virtual Function? What makes a class an "Abstract Class", and what are the restrictions on instantiating it?

Module 5 - Practical Lab Work

Lab 5.1: Shape Interface System

Define an abstract class `Shape` with a pure virtual function `double getArea()` and `void printName()`. Derive two classes, `Circle` (with a radius parameter) and `Rectangle` (with width and height parameters). Write a main function that creates an array of `Shape*` pointers holding both Circle and Rectangle objects, and prints their names and areas iteratively using dynamic polymorphism.